

School Management System

Project Report — Object-Oriented Programming

Student	ID
Ahmed Abu Yousef	2549011076
Ahmed Ekhzaiq	2549011068
Ahmed Alhajjar	2549011060

A. General Information

Project Description

The School Management System is a Java console application that digitizes core administrative tasks for a school: registering students and teachers, creating courses, enrolling students in courses, and tracking the school's finances — tuition fees collected from students and salaries paid to teachers. All data is written to a text file so nothing is lost between runs.

Objective

The project's goal is to replace manual, error-prone record keeping with a single program that keeps student, teacher, course, and financial records consistent, while giving the user a straightforward menu to interact with the system.

Main Features

- Interactive console menu covering 10 operations: add student, add teacher, add course, register a student in a course, pay fees, pay salary, list all people, list all courses, view financial status, and save & exit.
- Persistent storage: all records are saved to school.txt and reloaded automatically the next time the program runs.
- Financial ledger: fee payments and salary payments update a running total of money earned and spent, with validation against overpayment.
- Course management: courses are linked to the teacher who gives them, and students can be enrolled in one or more courses.
- Input validation: invalid menu choices or non-numeric input are caught and reported without crashing the program.

Challenges

- Keeping the financial totals (totalMoneyEarned / totalMoneySpent) consistent between a fresh run and a run that reloads existing data from the text file, since both the individual records and a separate FINANCE summary line have to agree.
- Designing the text file format so that courses, enrollments, and people could all be represented as plain lines while still being able to reconstruct object references between them (e.g. linking a Course back to its Teacher) on load.
- Deciding what belongs in the abstract Person class versus what is specific to Student or Teacher, since some behaviors (like paying fees or receiving a salary) only make sense for one of the two.

B. Programming Explanation

OOP Concepts Applied

- Encapsulation — all fields across Person, Student, Teacher, Course, and School are private, accessed only through public getters/setters.
- Inheritance — Student and Teacher both extend the abstract Person class, reusing ID, name, and date-of-birth handling.

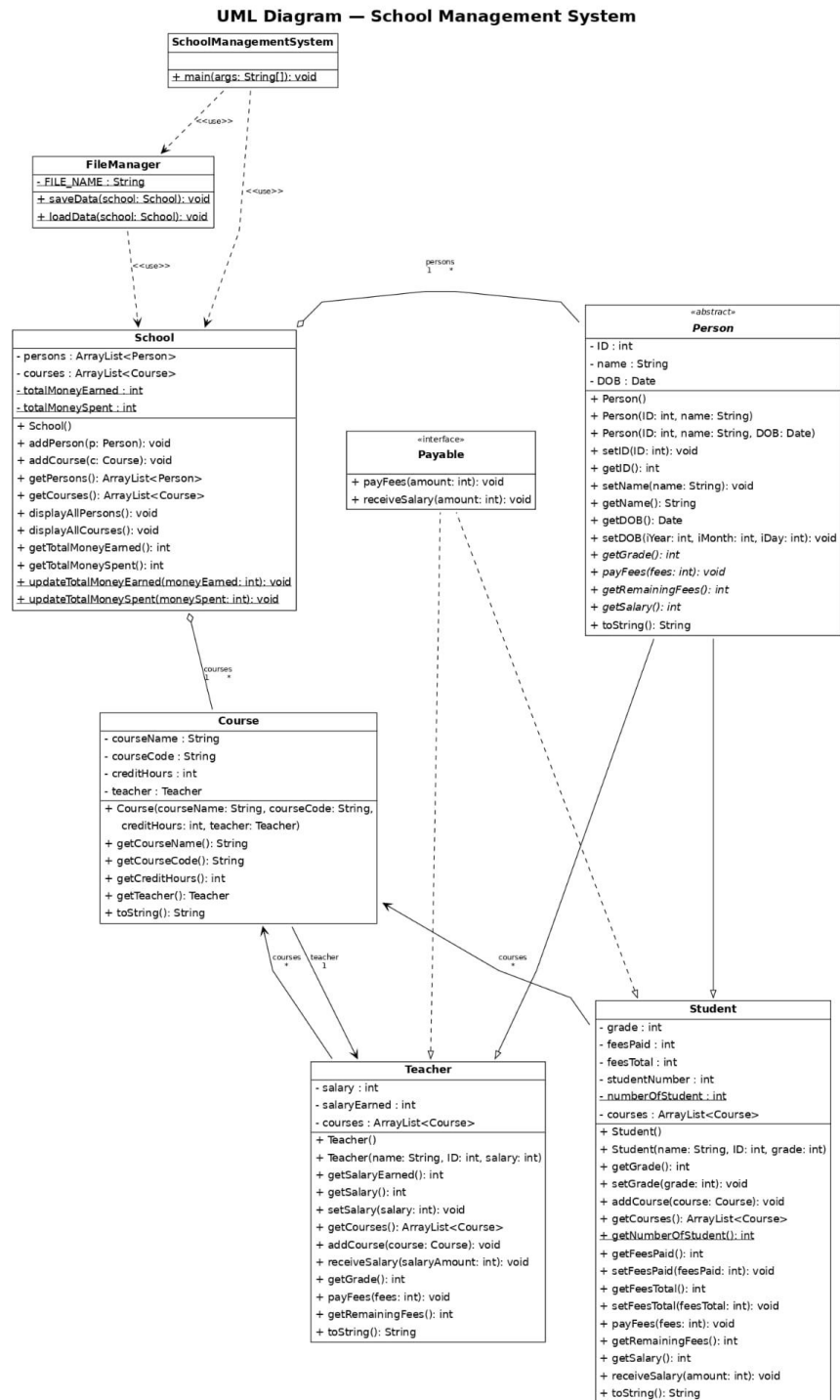
- Polymorphism — Student and Teacher each implement the Payable interface differently, and Person declares abstract methods (getGrade, payFees, getRemainingFees, getSalary) that each subclass overrides with its own behavior.
- Abstract Class — Person cannot be instantiated directly; it forces every subclass to define role-specific behavior.
- Interface — Payable defines a financial contract (payFees, receiveSalary) that both Student and Teacher implement, even though only one of the two methods is meaningful for each.
- Aggregation / Composition — School holds collections of Person and Course objects; Course holds a reference to the Teacher who teaches it; Student holds a list of enrolled Course objects.

Main Classes

Class	Responsibility
Person	Abstract base class for anyone in the school. Stores ID, name, and date of birth, and declares abstract methods that force Student and Teacher to define their own role-specific behavior.
Student	Extends Person and implements Payable. Tracks grade, fees paid/owed, and enrolled courses; pays fees and validates against overpayment.
Teacher	Extends Person and implements Payable. Tracks salary, total salary received, and the courses they teach.
Course	Represents a single course: name, code, credit hours, and the Teacher who teaches it.
Payable	Interface implemented by Student and Teacher, declaring payFees() and receiveSalary() as a shared financial contract.
School	Central aggregator holding every Person and Course in the system, plus the school-wide financial totals.
FileManager	Handles saving the entire school state to school.txt and reloading it on startup, using try-with-resources for safe file I/O.
SchoolManagementSystem	Entry point of the program; drives the console menu loop and dispatches each option to the right class.

UML Class Diagram

The diagram below shows the classes described above and how they relate: inheritance from Person, interface realization of Payable, and the aggregation/association links between School, Person, Course, and Teacher.



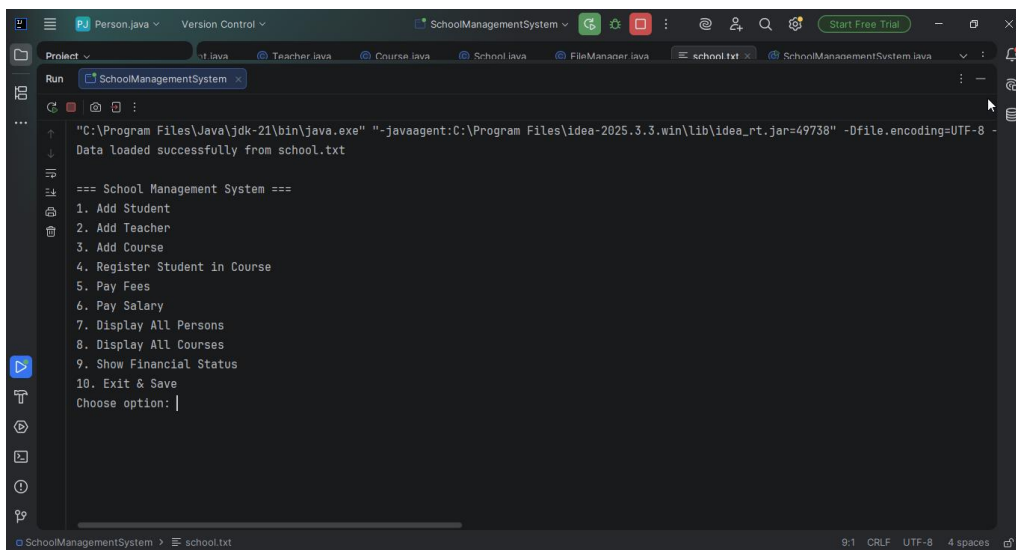
Video Presentation

https://drive.google.com/file/d/1EvDyVB93-1vI9ATBZcWGLajDt2JiM8J_/view?usp=drive_link

Link GitHub

<https://github.com/A7madEmad/School.Management.System.git>

Screenshot



```
Run SchoolManagementSystem
C:\Program Files\Java\jdk-21\bin\java.exe "-javaagent:C:\Program Files\Idea-2025.3.3.win\lib\idea_rt.jar=49738" -Dfile.encoding=UTF-8
Data loaded successfully from school.txt

=== School Management System ===
1. Add Student
2. Add Teacher
3. Add Course
4. Register Student in Course
5. Pay Fees
6. Pay Salary
7. Display All Persons
8. Display All Courses
9. Show Financial Status
10. Exit & Save
Choose option: |
```

```
=== School Management System ===
1. Add Student
2. Add Teacher
3. Add Course
4. Register Student in Course
5. Pay Fees
6. Pay Salary
7. Display All Persons
8. Display All Courses
9. Show Financial Status
10. Exit & Save
Choose option: 1
Enter student ID: 123
Enter student name: Hasan
Enter grade: 7
Enter DOB (year month day): 2007 5 17
Student added successfully!
```

```
=== School Management System ===
1. Add Student
2. Add Teacher
3. Add Course
4. Register Student in Course
5. Pay Fees
6. Pay Salary
7. Display All Persons
8. Display All Courses
9. Show Financial Status
10. Exit & Save
Choose option: 2
Enter teacher ID: 124
Enter teacher name: Kmal
Enter salary: 5000
Enter DOB (year month day): 1992 1 1
Teacher added successfully!
```

```
=== School Management System ===
1. Add Student
2. Add Teacher
3. Add Course
4. Register Student in Course
5. Pay Fees
6. Pay Salary
7. Display All Persons
8. Display All Courses
9. Show Financial Status
10. Exit & Save
Choose option: 3
Enter course name: Math
Enter course code: C2603
Enter credit hours: 25
Enter teacher name for this course: Kmal
✓ Course added successfully!
```

=== School Management System ===

1. Add Student
 2. Add Teacher
 3. Add Course
 4. Register Student in Course
 5. Pay Fees
 6. Pay Salary
 7. Display All Persons
 8. Display All Courses
 9. Show Financial Status
 10. Exit & Save
- Choose option: 7

=== All Persons ===

Student 1: name: Ahmed, ID: 12601, Date Of Birth:(Mon Feb 14 21:05:14 AST 2005), Grade: 6, Fees Paid: \$6000, Remaining Fees: \$0
Student 2: name: Alla, ID: 12602, Date Of Birth:(Mon Jan 05 21:06:04 AST 2004), Grade: 8, Fees Paid: \$5000, Remaining Fees: \$2000
Teacher: name: Mohammed, ID: 22601, Date Of Birth:(Sat Feb 13 21:09:26 AST 1999), Salary: \$2000, Salary Earned: \$4000
Teacher: name: Mr.Ali, ID: 22602, Date Of Birth:(Sat Feb 13 21:09:26 AST 1999), Salary: \$3000, Salary Earned: \$3000
Student 3: name: Ali, ID: 123, Date Of Birth:(Fri Mar 13 19:17:30 AST 2009), Grade: 3, Fees Paid: \$0, Remaining Fees: \$5000
Student 4: name: Hasan, ID: 123, Date Of Birth:(Thu May 17 20:04:07 AST 2007), Grade: 7, Fees Paid: \$0, Remaining Fees: \$7000
Teacher: name: Kmal, ID: 124, Date Of Birth:(Wed Jan 01 20:05:34 AST 1992), Salary: \$5000, Salary Earned: \$0

=== School Management System ===

1. Add Student
 2. Add Teacher
 3. Add Course
 4. Register Student in Course
 5. Pay Fees
 6. Pay Salary
 7. Display All Persons
 8. Display All Courses
 9. Show Financial Status
 10. Exit & Save
- Choose option: 8

=== All Courses ===

Course: Arabic (C2601), Credit Hours: 30, Taught by: Mohammed
Course: English (C2602), Credit Hours: 30, Taught by: Mr.Ali
Course: Math (C2603), Credit Hours: 25, Taught by: Kmal

```
=== School Management System ===
```

1. Add Student
2. Add Teacher
3. Add Course
4. Register Student in Course
5. Pay Fees
6. Pay Salary
7. Display All Persons
8. Display All Courses
9. Show Financial Status
10. Exit & Save

Choose option: 9

```
=== Financial Status ===
```

Total Money Earned: \$2000

Total Money Spent: \$0

Net Profit: \$2000

```
=== School Management System ===
```

1. Add Student
2. Add Teacher
3. Add Course
4. Register Student in Course
5. Pay Fees
6. Pay Salary
7. Display All Persons
8. Display All Courses
9. Show Financial Status
10. Exit & Save

Choose option: 10

Saving data...

Exiting... Goodbye!